In [1]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error
import matplotlib.pyplot as plt
import seaborn as sns
```

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
Cell In[1], line 5
      3 from sklearn.model_selection import train_test_split
      4 from sklearn.ensemble import RandomForestRegressor
----> 5 from xgboost import XGBRegressor
      6 from sklearn.metrics import mean_absolute_error, mean_squared_error
      7 import matplotlib.pyplot as plt

ModuleNotFoundError: No module named 'xgboost'
```

In [3]:
```
pip install xgboost
```

```
Collecting xgboost
  Downloading xgboost-3.0.0-py3-none-win_amd64.whl.metadata (2.1 kB)
Requirement already satisfied: numpy in c:\users\sande\anaconda3\lib\site-packages
(from xgboost) (1.26.4)
Requirement already satisfied: scipy in c:\users\sande\anaconda3\lib\site-packages
(from xgboost) (1.13.1)
Downloading xgboost-3.0.0-py3-none-win_amd64.whl (150.0 MB)
   ---------------------------------------- 0.0/150.0 MB ? eta -:--:--
    --------------------------------------- 2.4/150.0 MB 13.4 MB/s eta 0:00:11
   - -------------------------------------- 5.5/150.0 MB 14.0 MB/s eta 0:00:11
   - -------------------------------------- 7.3/150.0 MB 12.6 MB/s eta 0:00:12
   -- ------------------------------------- 8.7/150.0 MB 10.5 MB/s eta 0:00:14
   -- ------------------------------------- 10.7/150.0 MB 10.5 MB/s eta 0:00:14
   --- ------------------------------------ 13.4/150.0 MB 10.8 MB/s eta 0:00:13
   ---- ----------------------------------- 15.2/150.0 MB 10.6 MB/s eta 0:00:13
   ---- ----------------------------------- 16.3/150.0 MB 10.2 MB/s eta 0:00:14
   ---- ----------------------------------- 18.1/150.0 MB 9.8 MB/s eta 0:00:14
   ----- ---------------------------------- 19.9/150.0 MB 9.6 MB/s eta 0:00:14
   ----- ---------------------------------- 22.0/150.0 MB 9.5 MB/s eta 0:00:14
   ------ --------------------------------- 24.6/150.0 MB 9.9 MB/s eta 0:00:13
   ------- -------------------------------- 27.5/150.0 MB 10.1 MB/s eta 0:00:13
   -------- ------------------------------- 30.4/150.0 MB 10.4 MB/s eta 0:00:12
   -------- ------------------------------- 33.6/150.0 MB 10.7 MB/s eta 0:00:11
   --------- ------------------------------ 36.2/150.0 MB 10.9 MB/s eta 0:00:11
   ---------- ----------------------------- 39.6/150.0 MB 11.2 MB/s eta 0:00:10
   ----------- ---------------------------- 42.7/150.0 MB 11.4 MB/s eta 0:00:10
   ------------ --------------------------- 45.6/150.0 MB 11.5 MB/s eta 0:00:10
   ------------- -------------------------- 49.0/150.0 MB 11.8 MB/s eta 0:00:09
   ------------- -------------------------- 52.4/150.0 MB 11.9 MB/s eta 0:00:09
   -------------- ------------------------- 54.3/150.0 MB 11.9 MB/s eta 0:00:09
   -------------- ------------------------- 56.1/150.0 MB 11.7 MB/s eta 0:00:09
   --------------- ------------------------ 59.5/150.0 MB 11.8 MB/s eta 0:00:08
   ---------------- ----------------------- 63.2/150.0 MB 12.1 MB/s eta 0:00:08
   ----------------- ---------------------- 66.3/150.0 MB 12.2 MB/s eta 0:00:07
   ------------------ --------------------- 69.7/150.0 MB 12.3 MB/s eta 0:00:07
   ------------------- -------------------- 73.4/150.0 MB 12.5 MB/s eta 0:00:07
   -------------------- ------------------- 76.5/150.0 MB 12.6 MB/s eta 0:00:06
   -------------------- ------------------- 80.0/150.0 MB 12.8 MB/s eta 0:00:06
   --------------------- ------------------ 83.4/150.0 MB 12.8 MB/s eta 0:00:06
   ---------------------- ----------------- 87.0/150.0 MB 13.0 MB/s eta 0:00:05
   ----------------------- ---------------- 90.2/150.0 MB 13.1 MB/s eta 0:00:05
   ----------------------- ---------------- 93.6/150.0 MB 13.2 MB/s eta 0:00:05
   ------------------------ --------------- 97.0/150.0 MB 13.3 MB/s eta 0:00:04
   ------------------------- -------------- 100.4/150.0 MB 13.3 MB/s eta 0:00:04
   ------------------------- -------------- 103.5/150.0 MB 13.4 MB/s eta 0:00:04
   -------------------------- ------------- 106.7/150.0 MB 13.4 MB/s eta 0:00:04
   -------------------------- ------------- 110.1/150.0 MB 13.5 MB/s eta 0:00:03
   --------------------------- ------------ 113.5/150.0 MB 13.5 MB/s eta 0:00:03
   ---------------------------- ----------- 116.1/150.0 MB 13.6 MB/s eta 0:00:03
   ---------------------------- ----------- 118.8/150.0 MB 13.5 MB/s eta 0:00:03
   ----------------------------- ---------- 121.9/150.0 MB 13.5 MB/s eta 0:00:03
   ------------------------------ --------- 125.3/150.0 MB 13.6 MB/s eta 0:00:02
   ------------------------------ --------- 128.5/150.0 MB 13.6 MB/s eta 0:00:02
   ------------------------------- -------- 132.1/150.0 MB 13.7 MB/s eta 0:00:02
   -------------------------------- ------- 135.3/150.0 MB 13.7 MB/s eta 0:00:02
   -------------------------------- ------- 138.7/150.0 MB 13.7 MB/s eta 0:00:01
```

```
   ---------------------------------- -- 141.6/150.0 MB 13.8 MB/s eta 0:00:01
   ---------------------------------- - 145.2/150.0 MB 13.8 MB/s eta 0:00:01
   ----------------------------------  148.6/150.0 MB 13.9 MB/s eta 0:00:01
   ----------------------------------  149.9/150.0 MB 13.9 MB/s eta 0:00:01
   ---------------------------------- 150.0/150.0 MB 13.5 MB/s eta 0:00:00
Installing collected packages: xgboost
Successfully installed xgboost-3.0.0
Note: you may need to restart the kernel to use updated packages.
```

In [4]:
```
!pip install xgboost
```

```
Requirement already satisfied: xgboost in c:\users\sande\anaconda3\lib\site-packages
(3.0.0)
Requirement already satisfied: numpy in c:\users\sande\anaconda3\lib\site-packages
(from xgboost) (1.26.4)
Requirement already satisfied: scipy in c:\users\sande\anaconda3\lib\site-packages
(from xgboost) (1.13.1)
```

In [5]:
```
from xgboost import XGBRegressor
```

In [9]:
```
from xgboost import XGBRegressor
```

In [29]:
```
train = pd.read_csv(r"C:\Users\sande\Downloads\cltvdataset\train.csv")
test = pd.read_csv(r"C:\Users\sande\Downloads\cltvdataset\test.csv")
```

In [31]:
```
import pandas as pd

train = pd.read_csv("train.csv")
test = pd.read_csv("test.csv")
```

In [33]:
```
print(train.head())
print(train.columns)
```

```
   CustomerID  Customer.Lifetime.Value Coverage             Education  \
0        5917              7824.372789    Basic              Bachelor
1        2057              8005.964669    Basic               College
2        4119              8646.504109    Basic  High School or Below
3        1801              9294.088719    Basic               College
4        9618              5595.971365    Basic              Bachelor

  EmploymentStatus Gender Income Location.Geo Location.Code Marital.Status  \
0       Unemployed      F      0    17.7,77.7         Urban        Married
1         Employed      M  63357    28.8,76.6      Suburban        Married
2         Employed      F  64125    21.6,88.4         Urban        Married
3         Employed      M  67544      19,72.5      Suburban        Married
4          Retired      F  19651    19.1,74.7      Suburban        Married

   ...  Months.Since.Policy.Inception  Number.of.Open.Complaints  \
0  ...                             33                        NaN
1  ...                             42                        0.0
2  ...                             44                        0.0
3  ...                             15                        NaN
4  ...                             68                        0.0

   Number.of.Policies     Policy.Type        Policy Renew.Offer.Type  \
0                 2.0   Personal Auto   Personal L2           Offer2
1                 5.0   Personal Auto   Personal L2           Offer2
2                 3.0   Personal Auto   Personal L1           Offer2
3                 3.0  Corporate Auto  Corporate L3           Offer1
4                 5.0   Personal Auto   Personal L1           Offer2

  Sales.Channel Total.Claim.Amount  Vehicle.Class  Vehicle.Size
0        Branch         267.214383  Four-Door Car           2.0
1         Agent         565.508572            SUV           2.0
2        Branch         369.818708            SUV           1.0
3        Branch         556.800000            SUV           3.0
4           Web         345.600000   Two-Door Car           3.0

[5 rows x 22 columns]
Index(['CustomerID', 'Customer.Lifetime.Value', 'Coverage', 'Education',
       'EmploymentStatus', 'Gender', 'Income', 'Location.Geo', 'Location.Code',
       'Marital.Status', 'Monthly.Premium.Auto', 'Months.Since.Last.Claim',
       'Months.Since.Policy.Inception', 'Number.of.Open.Complaints',
       'Number.of.Policies', 'Policy.Type', 'Policy', 'Renew.Offer.Type',
       'Sales.Channel', 'Total.Claim.Amount', 'Vehicle.Class', 'Vehicle.Size'],
      dtype='object')
```

In [35]:
```python
X_train = train.drop(columns=["ltv"])  # Features only
y_train = train["ltv"]  # Target: lifetime value

X_test = test.copy()  # Features only for prediction
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[35], line 1
----> 1 X_train = train.drop(columns=["ltv"])  # Features only
      2 y_train = train["ltv"]  # Target: lifetime value
      4 X_test = test.copy()

File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:5581, in DataFrame.drop(sel
f, labels, axis, index, columns, level, inplace, errors)
   5433 def drop(
   5434     self,
   5435     labels: IndexLabel | None = None,
   (...)
   5442     errors: IgnoreRaise = "raise",
   5443 ) -> DataFrame | None:
   5444     """
   5445     Drop specified labels from rows or columns.
   5446
   (...)
   5579             weight  1.0     0.8
   5580     """
-> 5581     return super().drop(
   5582         labels=labels,
   5583         axis=axis,
   5584         index=index,
   5585         columns=columns,
   5586         level=level,
   5587         inplace=inplace,
   5588         errors=errors,
   5589     )

File ~\anaconda3\Lib\site-packages\pandas\core\generic.py:4788, in NDFrame.drop(sel
f, labels, axis, index, columns, level, inplace, errors)
   4786 for axis, labels in axes.items():
   4787     if labels is not None:
-> 4788         obj = obj._drop_axis(labels, axis, level=level, errors=errors)
   4790 if inplace:
   4791     self._update_inplace(obj)

File ~\anaconda3\Lib\site-packages\pandas\core\generic.py:4830, in NDFrame._drop_axi
s(self, labels, axis, level, errors, only_slice)
   4828         new_axis = axis.drop(labels, level=level, errors=errors)
   4829     else:
-> 4830         new_axis = axis.drop(labels, errors=errors)
   4831     indexer = axis.get_indexer(new_axis)
   4833 # Case for non-unique axis
   4834 else:

File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:7070, in Index.drop(s
elf, labels, errors)
   7068 if mask.any():
   7069     if errors != "ignore":
-> 7070         raise KeyError(f"{labels[mask].tolist()} not found in axis")
   7071     indexer = indexer[~mask]
   7072 return self.delete(indexer)
```

```
KeyError: "['ltv'] not found in axis"
```

In [37]:
```
print(train.columns.tolist())
train.head()
```

```
['CustomerID', 'Customer.Lifetime.Value', 'Coverage', 'Education', 'EmploymentStatu
s', 'Gender', 'Income', 'Location.Geo', 'Location.Code', 'Marital.Status', 'Monthly.
Premium.Auto', 'Months.Since.Last.Claim', 'Months.Since.Policy.Inception', 'Number.o
f.Open.Complaints', 'Number.of.Policies', 'Policy.Type', 'Policy', 'Renew.Offer.Typ
e', 'Sales.Channel', 'Total.Claim.Amount', 'Vehicle.Class', 'Vehicle.Size']
```

Out[37]:

	CustomerID	Customer.Lifetime.Value	Coverage	Education	EmploymentStatus	Gender
0	5917	7824.372789	Basic	Bachelor	Unemployed	F
1	2057	8005.964669	Basic	College	Employed	M
2	4119	8646.504109	Basic	High School or Below	Employed	F
3	1801	9294.088719	Basic	College	Employed	M
4	9618	5595.971365	Basic	Bachelor	Retired	F

5 rows × 22 columns

In [41]:
```
print(train.columns.tolist())
```

```
['CustomerID', 'Customer.Lifetime.Value', 'Coverage', 'Education', 'EmploymentStatu
s', 'Gender', 'Income', 'Location.Geo', 'Location.Code', 'Marital.Status', 'Monthly.
Premium.Auto', 'Months.Since.Last.Claim', 'Months.Since.Policy.Inception', 'Number.o
f.Open.Complaints', 'Number.of.Policies', 'Policy.Type', 'Policy', 'Renew.Offer.Typ
e', 'Sales.Channel', 'Total.Claim.Amount', 'Vehicle.Class', 'Vehicle.Size']
```

In [43]:
```
# Clean column names
train.columns = train.columns.str.strip().str.lower().str.replace(' ', '_')
test.columns = test.columns.str.strip().str.lower().str.replace(' ', '_')

print(train.columns.tolist())
```

```
['customerid', 'customer.lifetime.value', 'coverage', 'education', 'employmentstatu
s', 'gender', 'income', 'location.geo', 'location.code', 'marital.status', 'monthly.
premium.auto', 'months.since.last.claim', 'months.since.policy.inception', 'number.o
f.open.complaints', 'number.of.policies', 'policy.type', 'policy', 'renew.offer.typ
e', 'sales.channel', 'total.claim.amount', 'vehicle.class', 'vehicle.size']
```

In [47]:
```
X_train = train.drop(columns=["customer.lifetime.value"])
y_train = train["customer.lifetime.value"]

X_test = test.copy()
```

In [49]:
```
from xgboost import XGBRegressor

model = XGBRegressor(n_estimators=100, learning_rate=0.1, random_state=42)
model.fit(X_train, y_train)
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
File ~\anaconda3\Lib\site-packages\xgboost\data.py:407, in pandas_feature_info(data, meta, feature_names, feature_types, enable_categorical)
    406 try:
--> 407     new_feature_types.append(_pandas_dtype_mapper[dtype.name])
    408 except KeyError:

KeyError: 'object'

During handling of the above exception, another exception occurred:

ValueError                                Traceback (most recent call last)
Cell In[49], line 4
      1 from xgboost import XGBRegressor
      3 model = XGBRegressor(n_estimators=100, learning_rate=0.1, random_state=42)
----> 4 model.fit(X_train, y_train)

File ~\anaconda3\Lib\site-packages\xgboost\core.py:729, in require_keyword_args.<locals>.throw_if.<locals>.inner_f(*args, **kwargs)
    727 for k, arg in zip(sig.parameters, args):
    728     kwargs[k] = arg
--> 729 return func(**kwargs)

File ~\anaconda3\Lib\site-packages\xgboost\sklearn.py:1222, in XGBModel.fit(self, X, y, sample_weight, base_margin, eval_set, verbose, xgb_model, sample_weight_eval_set, base_margin_eval_set, feature_weights)
   1217 model, metric, params, feature_weights = self._configure_fit(
   1218     xgb_model, params, feature_weights
   1219 )
   1221 evals_result: TrainingCallback.EvalsLog = {}
-> 1222 train_dmatrix, evals = _wrap_evaluation_matrices(
   1223     missing=self.missing,
   1224     X=X,
   1225     y=y,
   1226     group=None,
   1227     qid=None,
   1228     sample_weight=sample_weight,
   1229     base_margin=base_margin,
   1230     feature_weights=feature_weights,
   1231     eval_set=eval_set,
   1232     sample_weight_eval_set=sample_weight_eval_set,
   1233     base_margin_eval_set=base_margin_eval_set,
   1234     eval_group=None,
   1235     eval_qid=None,
   1236     create_dmatrix=self._create_dmatrix,
   1237     enable_categorical=self.enable_categorical,
   1238     feature_types=self.feature_types,
   1239 )
   1241 if callable(self.objective):
   1242     obj: Optional[Objective] = _objective_decorator(self.objective)

File ~\anaconda3\Lib\site-packages\xgboost\sklearn.py:628, in _wrap_evaluation_matrices(missing, X, y, group, qid, sample_weight, base_margin, feature_weights, eval_set, sample_weight_eval_set, base_margin_eval_set, eval_group, eval_qid, create_dmatrix, enable_categorical, feature_types)
```

```
    607 def _wrap_evaluation_matrices(
    608     *,
    609     missing: float,
   (...)
    624     feature_types: Optional[FeatureTypes],
    625 ) -> Tuple[Any, List[Tuple[Any, str]]]:
    626     """Convert array_like evaluation matrices into DMatrix.  Perform validat
ion on the
    627     way."""
--> 628     train_dmatrix = create_dmatrix(
    629         data=X,
    630         label=y,
    631         group=group,
    632         qid=qid,
    633         weight=sample_weight,
    634         base_margin=base_margin,
    635         feature_weights=feature_weights,
    636         missing=missing,
    637         enable_categorical=enable_categorical,
    638         feature_types=feature_types,
    639         ref=None,
    640     )
    642     n_validation = 0 if eval_set is None else len(eval_set)
    644     def validate_or_none(meta: Optional[Sequence], name: str) -> Sequence:

File ~\anaconda3\Lib\site-packages\xgboost\sklearn.py:1137, in XGBModel._create_dmat
rix(self, ref, **kwargs)
   1135 if _can_use_qdm(self.tree_method, self.device) and self.booster != "gblinea
r":
   1136     try:
-> 1137         return QuantileDMatrix(
   1138             **kwargs, ref=ref, nthread=self.n_jobs, max_bin=self.max_bin
   1139         )
   1140     except TypeError:  # `QuantileDMatrix` supports lesser types than DMatri
x
   1141         pass

File ~\anaconda3\Lib\site-packages\xgboost\core.py:729, in require_keyword_args.<loc
als>.throw_if.<locals>.inner_f(*args, **kwargs)
    727 for k, arg in zip(sig.parameters, args):
    728     kwargs[k] = arg
--> 729 return func(**kwargs)

File ~\anaconda3\Lib\site-packages\xgboost\core.py:1614, in QuantileDMatrix.__init__
(self, data, label, weight, base_margin, missing, silent, feature_names, feature_typ
es, nthread, max_bin, ref, group, qid, label_lower_bound, label_upper_bound, feature
_weights, enable_categorical, max_quantile_batches, data_split_mode)
   1594     if any(
   1595         info is not None
   1596         for info in (
   (...)
   1607         )
   1608     ):
   1609         raise ValueError(
   1610             "If data iterator is used as input, data like label should be "
   1611             "specified as batch argument."
```

```
   1612         )
-> 1614 self._init(
   1615     data,
   1616     ref=ref,
   1617     label=label,
   1618     weight=weight,
   1619     base_margin=base_margin,
   1620     group=group,
   1621     qid=qid,
   1622     label_lower_bound=label_lower_bound,
   1623     label_upper_bound=label_upper_bound,
   1624     feature_weights=feature_weights,
   1625     feature_names=feature_names,
   1626     feature_types=feature_types,
   1627     enable_categorical=enable_categorical,
   1628     max_quantile_blocks=max_quantile_batches,
   1629 )

File ~\anaconda3\Lib\site-packages\xgboost\core.py:1678, in QuantileDMatrix._init(se
lf, data, ref, enable_categorical, max_quantile_blocks, **meta)
   1663 config = make_jcargs(
   1664     nthread=self.nthread,
   1665     missing=self.missing,
   1666     max_bin=self.max_bin,
   1667     max_quantile_blocks=max_quantile_blocks,
   1668 )
   1669 ret = _LIB.XGQuantileDMatrixCreateFromCallback(
   1670     None,
   1671     it.proxy.handle,
   (...)
   1676     ctypes.byref(handle),
   1677 )
-> 1678 it.reraise()
   1679 # delay check_call to throw intermediate exception first
   1680 _check_call(ret)

File ~\anaconda3\Lib\site-packages\xgboost\core.py:572, in DataIter.reraise(self)
    570 exc = self._exception
    571 self._exception = None
--> 572 raise exc

File ~\anaconda3\Lib\site-packages\xgboost\core.py:553, in DataIter._handle_exceptio
n(self, fn, dft_ret)
    550     return dft_ret
    552 try:
--> 553     return fn()
    554 except Exception as e:  # pylint: disable=broad-except
    555     # Defer the exception in order to return 0 and stop the iteration.
    556     # Exception inside a ctype callback function has no effect except
    557     # for printing to stderr (doesn't stop the execution).
    558     tb = sys.exc_info()[2]

File ~\anaconda3\Lib\site-packages\xgboost\core.py:640, in DataIter._next_wrapper.<l
ocals>.<lambda>()
    638     self._temporary_data = None
    639 # pylint: disable=not-callable
```

```
--> 640 return self._handle_exception(lambda: int(self.next(input_data)), 0)

File ~\anaconda3\Lib\site-packages\xgboost\data.py:1654, in SingleBatchInternalIter.
next(self, input_data)
   1652     return False
   1653 self.it += 1
-> 1654 input_data(**self.kwargs)
   1655 return True

File ~\anaconda3\Lib\site-packages\xgboost\core.py:729, in require_keyword_args.<loc
als>.throw_if.<locals>.inner_f(*args, **kwargs)
    727 for k, arg in zip(sig.parameters, args):
    728     kwargs[k] = arg
--> 729 return func(**kwargs)

File ~\anaconda3\Lib\site-packages\xgboost\core.py:620, in DataIter._next_wrapper.<l
ocals>.input_data(data, feature_names, feature_types, **kwargs)
    618     new, cat_codes, feature_names, feature_types = self._temporary_data
    619 else:
--> 620     new, cat_codes, feature_names, feature_types = _proxy_transform(
    621         data,
    622         feature_names,
    623         feature_types,
    624         self._enable_categorical,
    625     )
    626 # Stage the data, meta info are copied inside C++ MetaInfo.
    627 self._temporary_data = (new, cat_codes, feature_names, feature_types)

File ~\anaconda3\Lib\site-packages\xgboost\data.py:1707, in _proxy_transform(data, f
eature_names, feature_types, enable_categorical)
   1705     return df_pa, None, feature_names, feature_types
   1706 if _is_pandas_df(data):
-> 1707     df, feature_names, feature_types = _transform_pandas_df(
   1708         data, enable_categorical, feature_names, feature_types
   1709     )
   1710     return df, None, feature_names, feature_types
   1711 raise TypeError("Value type is not supported for data iterator:" + str(type
(data)))

File ~\anaconda3\Lib\site-packages\xgboost\data.py:640, in _transform_pandas_df(dat
a, enable_categorical, feature_names, feature_types, meta)
    637 if meta and len(data.columns) > 1 and meta not in _matrix_meta:
    638     raise ValueError(f"DataFrame for {meta} cannot have multiple columns")
--> 640 feature_names, feature_types = pandas_feature_info(
    641     data, meta, feature_names, feature_types, enable_categorical
    642 )
    644 arrays = pandas_transform_data(data)
    645 return PandasTransformed(arrays), feature_names, feature_types

File ~\anaconda3\Lib\site-packages\xgboost\data.py:409, in pandas_feature_info(data,
meta, feature_names, feature_types, enable_categorical)
    407             new_feature_types.append(_pandas_dtype_mapper[dtype.name])
    408         except KeyError:
--> 409             _invalid_dataframe_dtype(data)
    411 if feature_types is None and meta is None:
    412     feature_types = new_feature_types
```

```
File ~\anaconda3\Lib\site-packages\xgboost\data.py:372, in _invalid_dataframe_dtype
(data)
    370 type_err = "DataFrame.dtypes for data must be int, float, bool or category."
    371 msg = f"""{type_err} {_ENABLE_CAT_ERR} {err}"""
--> 372 raise ValueError(msg)

ValueError: DataFrame.dtypes for data must be int, float, bool or category. When cat
egorical type is supplied, the experimental DMatrix parameter`enable_categorical` mu
st be set to `True`.  Invalid columns:coverage: object, education: object, employmen
tstatus: object, gender: object, income: object, location.geo: object, location.cod
e: object, marital.status: object, policy.type: object, policy: object, renew.offer.
type: object, sales.channel: object, vehicle.class: object
```

In [52]:
```
print(X_train.dtypes)
```

```
customerid                      int64
coverage                       object
education                      object
employmentstatus               object
gender                         object
income                         object
location.geo                   object
location.code                  object
marital.status                 object
monthly.premium.auto          float64
months.since.last.claim         int64
months.since.policy.inception   int64
number.of.open.complaints     float64
number.of.policies            float64
policy.type                    object
policy                         object
renew.offer.type               object
sales.channel                  object
total.claim.amount            float64
vehicle.class                  object
vehicle.size                  float64
dtype: object
```

In [54]:
```
X_train_encoded = pd.get_dummies(X_train)
X_test_encoded = pd.get_dummies(X_test)

# Align columns in test set to match train set
X_test_encoded = X_test_encoded.reindex(columns=X_train_encoded.columns, fill_value
```

In [63]:
```
X_test["predicted_ltv"] = predictions
X_test["segment"] = pd.qcut(X_test["predicted_ltv"], q=4, labels=["Low", "Mid", "Hi

X_test[["customer_id", "predicted_ltv", "segment"]].to_csv("ltv_predictions.csv", i
```

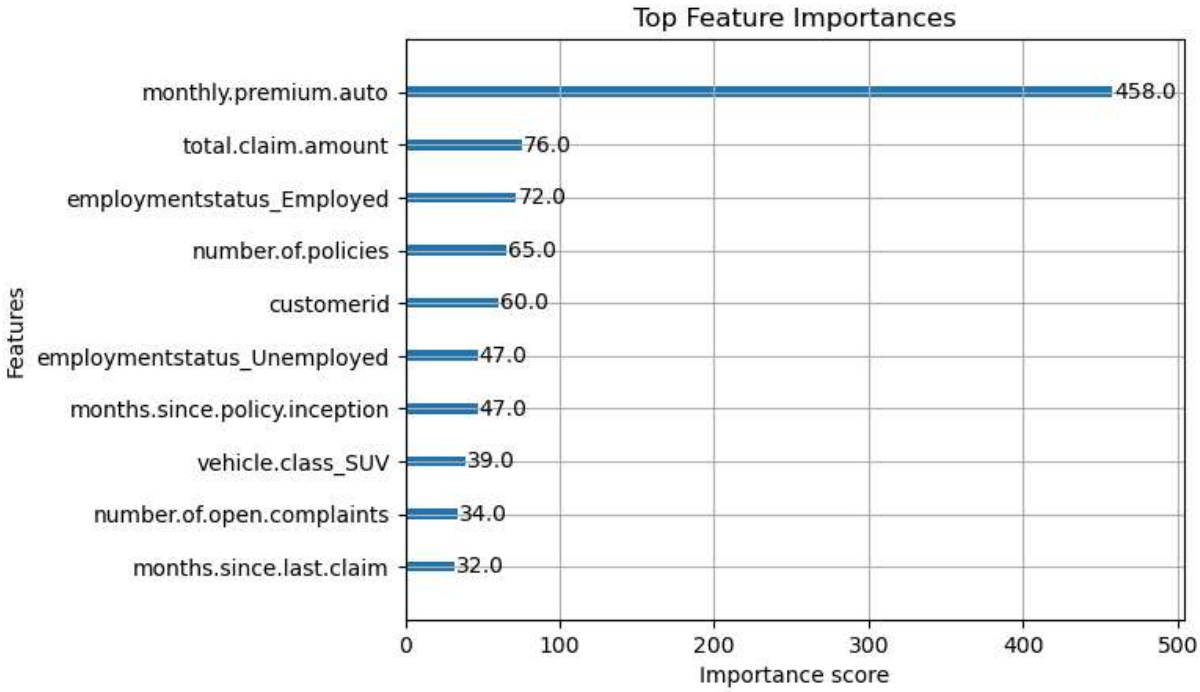
```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[63], line 4
      1 X_test["predicted_ltv"] = predictions
      2 X_test["segment"] = pd.qcut(X_test["predicted_ltv"], q=4, labels=["Low", "Mi
d", "High", "Top"])
----> 4 X_test[["customer_id", "predicted_ltv", "segment"]].to_csv("ltv_predictions.
csv", index=False)

File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:4108, in DataFrame.__getitem
__(self, key)
   4106     if is_iterator(key):
   4107         key = list(key)
-> 4108     indexer = self.columns._get_indexer_strict(key, "columns")[1]
   4110 # take() does not accept boolean indexers
   4111 if getattr(indexer, "dtype", None) == bool:

File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:6200, in Index._get_i
ndexer_strict(self, key, axis_name)
   6197 else:
   6198     keyarr, indexer, new_indexer = self._reindex_non_unique(keyarr)
-> 6200 self._raise_if_missing(keyarr, indexer, axis_name)
   6202 keyarr = self.take(indexer)
   6203 if isinstance(key, Index):
   6204     # GH 42790 - Preserve name from an Index

File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:6252, in Index._raise
_if_missing(self, key, indexer, axis_name)
   6249     raise KeyError(f"None of [{key}] are in the [{axis_name}]")
   6251 not_found = list(ensure_index(key)[missing_mask.nonzero()[0]].unique())
-> 6252 raise KeyError(f"{not_found} not in index")

KeyError: "['customer_id'] not in index"
```

In [65]:
```
X_test["predicted_ltv"] = predictions
X_test["segment"] = pd.qcut(X_test["predicted_ltv"], q=4, labels=["Low", "Mid", "Hi

X_test[["customerid", "predicted_ltv", "segment"]].to_csv("ltv_predictions.csv", in
```

In [68]:
```
pd.DataFrame({"predicted_ltv": predictions}).to_csv("ltv_predictions.csv", index=Fa
```

In [70]:
```
import matplotlib.pyplot as plt
from xgboost import plot_importance

plt.figure(figsize=(10, 6))
plot_importance(model, max_num_features=10)  # Top 10 features
plt.title("Top Feature Importances")
plt.show()
```

```
<Figure size 1000x600 with 0 Axes>
```



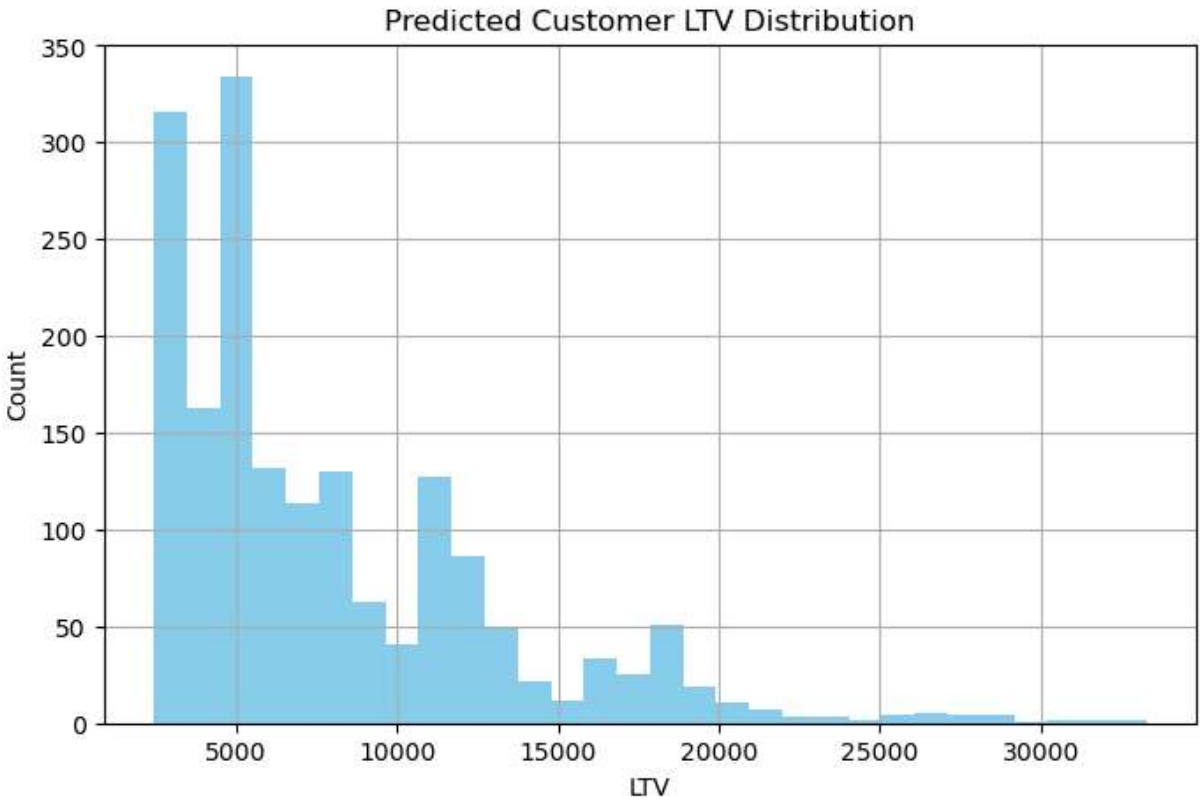
In [73]:
```
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

# If you have actual test targets
# y_test = test["ltv"]
# mae = mean_absolute_error(y_test, predictions)
# rmse = np.sqrt(mean_squared_error(y_test, predictions))

# print("MAE:", mae)
# print("RMSE:", rmse)
```

In [78]:
```
import joblib

joblib.dump(model, "xgb_ltv_model.pkl")  # Save model
```

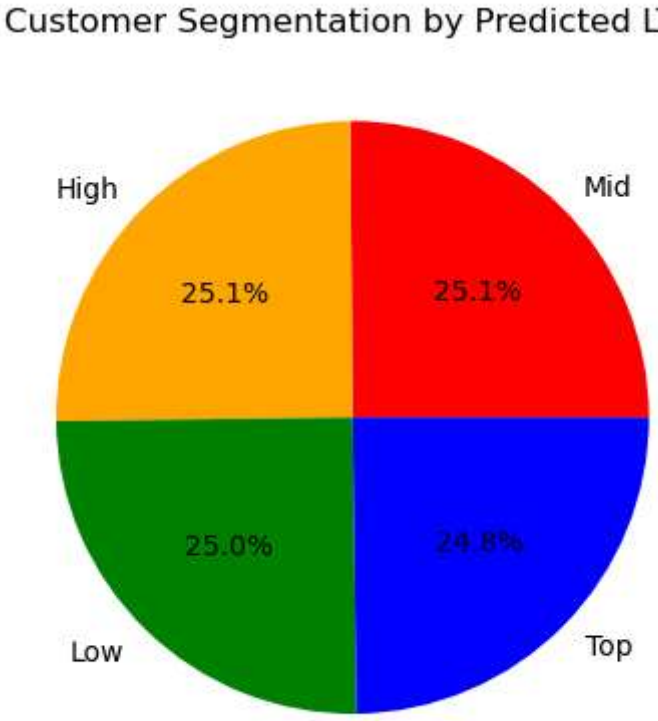
Out[78]:
```
['xgb_ltv_model.pkl']
```

In [80]:
```
model = joblib.load("xgb_ltv_model.pkl")
```

In [82]:
```
plt.figure(figsize=(8,5))
plt.hist(predictions, bins=30, color='skyblue')
plt.title("Predicted Customer LTV Distribution")
plt.xlabel("LTV")
plt.ylabel("Count")
plt.grid(True)
plt.show()
```



In [84]:
```
X_test["segment"].value_counts().plot.pie(autopct='%1.1f%%', colors=['red', 'orange
plt.title("Customer Segmentation by Predicted LTV")
plt.ylabel("")
plt.show()
```



In [88]:
```
train.head()
train.columns
```

Out[88]:
```
Index(['customerid', 'customer.lifetime.value', 'coverage', 'education',
       'employmentstatus', 'gender', 'income', 'location.geo', 'location.code',
       'marital.status', 'monthly.premium.auto', 'months.since.last.claim',
       'months.since.policy.inception', 'number.of.open.complaints',
       'number.of.policies', 'policy.type', 'policy', 'renew.offer.type',
       'sales.channel', 'total.claim.amount', 'vehicle.class', 'vehicle.size'],
      dtype='object')
```

In [ ]:
```

```